

Project Design Phase-II
Solution Requirements (Functional & Non-functional)

Date	28 SEPTEMBER 2026
Team ID	SWTID-2026-4636
Project Name	AI BLOGNEST API
Maximum Marks	4 Marks

Functional Requirements

Following are the functional requirements of the proposed solution.

FR No.	Functional Requirement (Epic)	Sub Requirement (Story / Sub-Task)
FR-1	User Registration & Authentication	User registration through form User login using email and password Generate authentication token Access protected APIs using token
FR-2	AI Blog Generation	Generate blog content using topic and category Generate AI-based blog title suggestions Generate relevant blog content
FR-3	AI Blog Summarization	Submit blog content for summarization Generate concise AI summary Return summary through API response
FR-4	Blog Management	Create and save blogs View all blogs View blog by ID Update blog Delete blog
FR-5	Blog Search & Categories	Search blogs using keywords Organize blogs by category Retrieve blogs based on category
FR-6	Input Validation & Error Handling	Validate required fields Handle invalid authentication Return appropriate error messages and HTTP status codes
FR-7	API Documentation & Testing	Document API endpoints Test APIs using Postman Verify request and response formats

Non-Functional Requirements

Following are the non-functional requirements of the proposed solution.

NFR No.	Non-Functional Requirement	Description
NFR-1	Usability	The API should be simple to use, with clear endpoints, request formats and meaningful response messages.
NFR-2	Security	User passwords should be securely handled, authentication should use tokens, and protected APIs should not be accessible without valid authentication.
NFR-3	Reliability	The API should provide consistent responses and handle invalid requests and

		server errors appropriately.
NFR-4	Performance	API requests should be processed within a reasonable response time, including AI content generation and summarization requests.
NFR-5	Availability	The API should be available whenever the server and required services are running and should handle requests without unnecessary downtime.
NFR-6	Scalability	The system should be designed so that additional users, blogs, categories and AI features can be added without major changes to the existing architecture.
NFR-7	Maintainability	The backend should use a structured architecture with separate routes, controllers, middleware and database components to simplify maintenance and future updates.
NFR-8	Compatibility	The API should support standard HTTP methods and JSON-based request and response formats so it can be used by web or mobile clients.